VINUNIVERSITY

Image Segmentation

AICB-P2T4 · Ngày 05 · Từ pixel đến quyết định, từ U-Net số 0 đến AI hỗ trợ

Giảng viên

VinUniversity · Data Track

HÃY SUY NGHĨ...

“Một quả dâu bị dập chỉ chiếm 4% diện tích ảnh. Box có thể nói “có dâu”. Mask mới chỉ đúng phần cần bỏ.”

Giữ câu hỏi này trong đầu khi học bài hôm nay

Nội dung bài học

1. Mask Và Quyết Định
2. Từ Tensor Đến U-Net
3. Dữ Liệu Dạy Segmentation
4. Đo Mask Và Audit Thất Bại
5. Ba Đường Đến Mask
6. Thực Hành Bốn Giờ

Hôm nay tạo được gì?

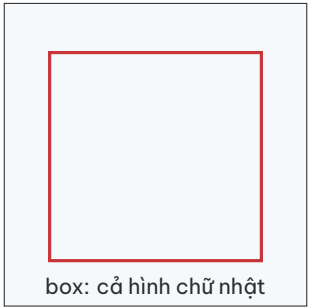
1. Phân biệt semantic, instance và panoptic segmentation bằng đầu ra thật.
2. Đọc tensor ảnh, mask, logits; tự xây một U-Net nhỏ từ trọng số ngẫu nhiên.
3. Đo IoU, Dice; tìm lỗi dữ liệu khiến điểm cao mà sản phẩm vẫn hỏng.
4. So sánh ba đường: U-Net tự train, YOLO segmentation fine-tune, SAM gợi ý mask.
5. Chạy dự án năm trạm trên ảnh mặt bàn: Gather $\rightarrow$ Label $\rightarrow$ Train $\rightarrow$ Infer $\rightarrow$ Decide.

Mask Và Quyết Định

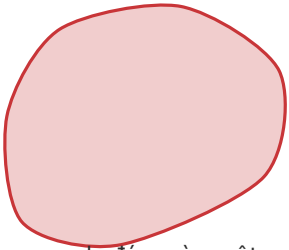
01

Segmentation trả lời “pixel nào thuộc về cái gì?”

Box đủ cho “có gì”, chưa đủ cho “ở đâu”



box: cả hình chữ nhật

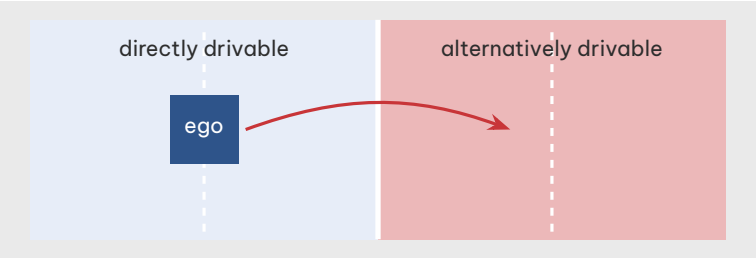


mask: đúng vùng vật

Quyết định thay đổi

Đếm vật: box có thể đủ. Cắt nền, đo diện tích, tìm vết bệnh, điều khiển robot gắp: cần mask.

Cùng là mặt đường, nhãn phụ thuộc ngữ cảnh



Trong contract **BDD100K**, hai dải kề nhau có vai trò chức năng khác nhau. Mask là bằng chứng ngữ cảnh, không phải chỉ dẫn "được phép lái".

Pixel, mask và lớp: ba từ phải chốt

Pixel

Một ô màu trong ảnh. Ảnh $H \times W$ có $H \cdot W$ vị trí.

Mask

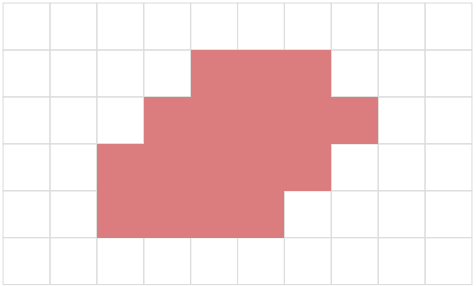
Một bản đồ cùng chiều cao, chiều rộng; mỗi vị trí mang đáp án.

Class

Tên ý nghĩa: nền, quả tốt, vùng dập. Số id chỉ là cách mã hoá.

Ảnh $[3, H, W] \longrightarrow$ mask nhị phân $[1, H, W]$ hoặc mask đa lớp $[H, W]$.

Một mask nhị phân chỉ có hai đáp án



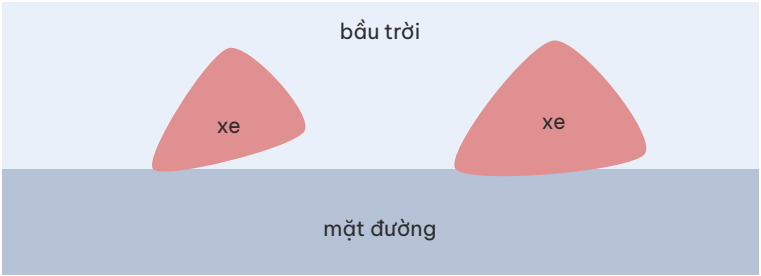
1: vùng quan tâm

0: nền

Không có “gần 1” trong ground truth nhị phân.

Sắc xám thường là lỗi re-size hoặc anti-alias.

Semantic: mọi pixel cùng lớp dùng một màu



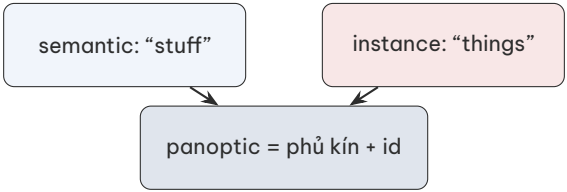
Hai chiếc xe nhập vào cùng lớp “xe”; mask không nói chiếc nào là chiếc nào.

Instance: mỗi vật có mask riêng



Cùng class “xe”, nhưng hai mask và hai id khác nhau.

Panoptic: mọi pixel có class, vật đếm được có id



Đường và trời: class. Người và xe: class + instance id. Không để pixel “vô chủ”.

Cùng một cảnh, ba hợp đồng đầu ra



Hình học hai xe và mặt đường giống hệt; output đổi theo câu hỏi sản phẩm.

Ba loại segmentation: ba tầng hiểu

Ý tưởng: Câu hỏi đầu ra quyết định cấu trúc nhãn.

●○○ **Cơ bản**	Semantic: pixel này thuộc lớp nào? Instance: pixel này thuộc vật nào?
●●○ **Nâng cao**	Panoptic ghép "stuff" và "things" thành một phân vùng kín của ảnh.
●●● **Chuyên sâu**	Đầu ra quyết định loss và metric: class map, tập mask, hay cặp semantic–instance cho mọi pixel.

Cảnh mặt bàn: quyết định trước, model sau

Bài toán

Camera nhìn khay trái cây. Hệ thống phải ước lượng phần diện tích bị dập và bật đèn kiểm tra nếu vượt ngưỡng.

Hợp đồng đầu ra

Một mask nhị phân “vùng dập”, cùng kích thước ảnh; quyết định dùng tỉ lệ pixel dập trên pixel quả.

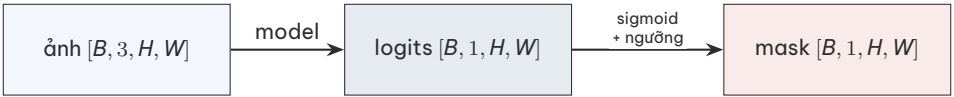
Đây là ví dụ hư cấu để học quy trình; ngưỡng quyết định phải được chọn bằng dữ liệu của ứng dụng thật.

Từ Tensor Đến U-Net

02

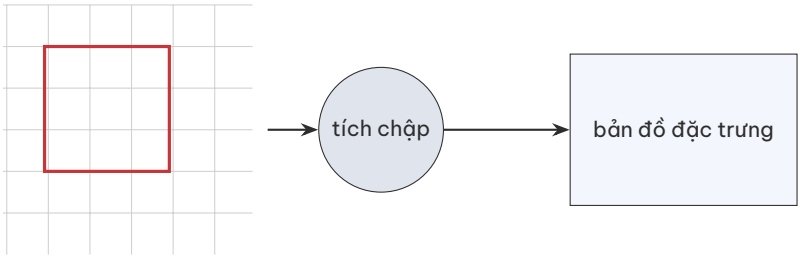
Một pixel đi qua CNN, encoder, decoder và trở thành logit

Tensor: chiếc hộp số của model



B là số ảnh trong một lô; 3 là ba kênh màu. Trục phải khớp trước khi học.

CNN: cùng một bộ lọc quét khắp ảnh



Filter học mẫu cạnh, màu, kết cấu; dùng lại cùng trọng số ở mọi vị trí.

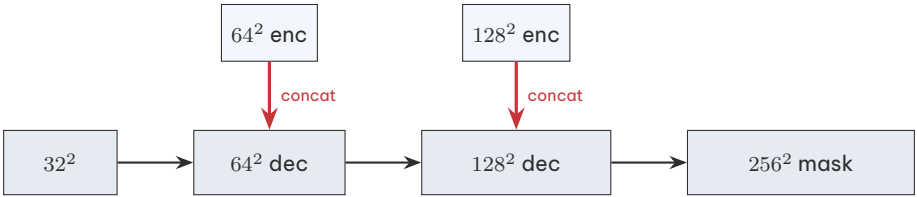
Encoder nén “ở đâu” để hiểu “là gì”



Đổi chác

Kích thước không gian nhỏ dần, số kênh thường tăng. Model có thêm ngữ cảnh nhưng dễ mất biên nhỏ.

Decoder phóng lại, skip connection trả chi tiết



Skip nối encoder và decoder ở **cùng độ phân giải**; decoder nhận ngữ cảnh lẫn chi tiết.

Logit: điểm thô trước xác suất

logit $z = -2, 0, 2$

↓ sigmoid

xác suất $0{,}12; 0{,}50; 0{,}88$

Vì sao loss nhận logits?

`BCEWithLogitsLoss` gộp sigmoid và binary cross entropy theo cách số học ổn định. Không sigmoid trước loss.

Chỉ dùng sigmoid khi cần xác suất để đo, hiển thị hoặc đặt ngưỡng.

U-Net nhỏ: hợp đồng trước kiến trúc


```
class TinyUNet(nn.Module):
  def __init__(self):
    super().__init__()
    self.enc1 = block(3, 16)
    self.pool = nn.MaxPool2d(2)
    self.enc2 = block(16, 32)
    self.up = nn.ConvTranspose2d(32, 16, 2, 2)
    self.dec = block(32, 16)
    self.head = nn.Conv2d(16, 1, 1)
```

Hai convolution tạo một “block”


```
def block(cin, cout):
  return nn.Sequential(
    nn.Conv2d(cin, cout, 3, padding=1),
    nn.ReLU(inplace=True),
    nn.Conv2d(cout, cout, 3, padding=1),
    nn.ReLU(inplace=True),
  )
```

padding=1 giữ H, W. ReLU thêm phi tuyến. Trọng số do PyTorch khởi tạo ngẫu nhiên; chưa học gì về trái cây.

Forward: nén, nối, giải mã

```
def forward(self, x):
  skip = self.enc1(x)
  low = self.enc2(self.pool(skip))
  up = self.up(low)
  fused = torch.cat([up, skip], dim=1)
  return self.head(self.dec(fused))
```

Nếu `x.shape == [4,3,128,128]` thì logits phải là `[4,1,128,128]`.

Một bước học: loss, gradient, cập nhật

```
model.train()
optimizer.zero_grad()
logits = model(images)
loss = criterion(logits, masks.float())
loss.backward()
optimizer.step()
```

Dấu hiệu tối thiểu của “đang học”: loss hữu hạn, gradient khác 0, ít nhất một tham số thay đổi.

Khởi tạo đầy đủ: random, loss, optimizer


```
import torch
from torch import nn
torch.manual_seed(7)
model = TinyUNet()
criterion = nn.BCEWithLogitsLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
```

Không tải checkpoint: mọi layer bắt đầu từ khởi tạo ngẫu nhiên của PyTorch. Learning rate là giá trị smoke test, không phải default sản phẩm.

U-Net: ba tầng hiểu

Ý tưởng: Nén để lấy ngữ cảnh, nối tắt để lấy lại vị trí.

●○○ Cơ bản	Encoder thu nhỏ; decoder phóng lớn; head trả một logit cho mỗi pixel.
●●○ Nâng cao	Skip connection nối đặc trưng cùng độ phân giải để decoder lấy lại chi tiết biên.
●●● Chuyên sâu	Tensor contract quyết định số kênh; loss trên logits tạo gradient đi qua encoder, decoder và skip path.

Từ FCN đến Transformer: mỗi đời sửa một nút

Kiến trúc	Ý tưởng nên nhớ
FCN (2015)	pixel-to-pixel; nối dự đoán thô với đặc trưng mịn
U-Net (2015)	encoder–decoder đối xứng, skip connection cho định vị
DeepLabv3+ (2018)	atrous context nhiều tỉ lệ + decoder sửa biên
SegFormer (2021)	encoder Transformer phân cấp + decoder MLP nhẹ
Mask2Former (2022)	masked attention cho ba loại segmentation

Train, validation, test: ba vai trò

Tập	Model được phép làm gì?	Ta dùng để làm gì?
Train	cập nhật trọng số	học mẫu từ ảnh và mask
Validation	không cập nhật	chọn epoch, threshold, cấu hình
Test	chỉ dùng ở cuối	ước lượng kết quả chưa bị chọn theo

Luật giữ kín

Nếu xem test rồi sửa model, tập đó đã trở thành validation. Cần một test mới để báo cáo cuối.

Bài tập nhanh (20 phút): lần theo một tensor

Với `images=[8,3,256,256]` và mask nhị phân, hãy trả lời:

1. Shape của mask trước loss?
2. Shape sau pool đầu tiên?
3. Vì sao head có đúng 1 kênh?
4. Dòng nào cập nhật trọng số?
5. Chỗ nào được phép sigmoid?

Làm cá nhân 7 phút, ghép đôi 6 phút, cả lớp đối chiếu 7 phút.

Sau giờ nghỉ: dữ liệu nào làm mask đáng tin?

Hãy giữ lại một câu hỏi về tensor hoặc U-Net.

Dữ Liệu Dạy Segmentation

Mask đúng nghĩa, đúng biên, đúng cặp và đúng cách chia

Năm trạm: hợp đồng xuyên suốt



Mỗi mũi tên là một hợp đồng dữ liệu. Hỏng shape, class id hay split ở đầu sẽ xuất hiện thành lỗi quyết định ở cuối.

Trạm 1 — Gather

Đầu vào

Bộ 20 ảnh học tập của khay trái cây: đổi quả, nền, góc và ánh sáng.

Đầu ra có thể kiểm

Manifest có image_id, group_id, nguồn và điều kiện. 20 ảnh chỉ đủ chạy vòng học, không chứng minh sẵn sàng sản xuất.

Đầu vào

Ảnh và quy tắc: thế nào là vùng dập; biên mơ hồ xử lý ra sao.

Đầu ra có thể kiểm

Mask PNG lossless; 0=nền, 1=vùng dập; mỗi ảnh có đúng một mask cùng kích thước.

Trạm 3 — Train

Đầu vào

Cặp ảnh–mask của train; cấu hình và seed.

Đầu ra có thể kiểm

Checkpoint, learning curve, metric validation theo từng epoch.

Đầu vào

Checkpoint và ảnh chưa thấy.

Đầu ra có thể kiểm

Logits/xác suất; mask sau threshold; thời gian và metadata của lần chạy.

Trạm 5 — Decide

Đầu vào

Mask dự đoán + vùng quả + luật sản phẩm.

Đầu ra có thể kiểm

Tỉ lệ vùng dập, quyết định, ca bị từ chối và log để audit.

Một cặp ảnh–mask phải khớp tuyệt đối

Khớp

Cùng stem hoặc id trong manifest; cùng H, W; crop/flip/rotate dùng cùng tham số.

Sai thầm lặng

Sắp tên theo chữ khiến `img10` đứng trước `img2`; ảnh transform nhưng mask không transform.

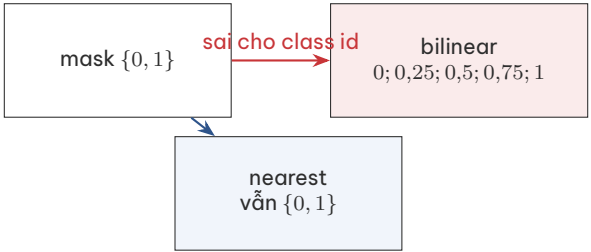
Không ghép hai thư mục chỉ bằng `zip(sorted(...))` nếu chưa kiểm id và số lượng.

Paired transform: ngẫu nhiên một lần, áp dụng hai lần

```
if random.random() < 0.5:
  image = TF.hflip(image)
  mask = TF.hflip(mask)
image = TF.resize(image, size, antialias=True)
mask = TF.resize(mask, size, interpolation=NEAREST)
```

Ảnh dùng nội suy mượt; mask class id dùng nearest-neighbor.

Resize mask sai sẽ sinh lớp không tồn tại



Polygon, lỗ và vật chạm biên

Polygon

Chuỗi điểm mô tả biên. Tự cắt nhau có thể rasterize khác nhau giữa thư viện.

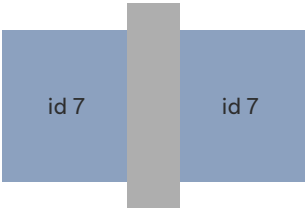
Hole

Vùng nền nằm trong vật phải được biểu diễn rõ; một contour chưa chắc đủ.

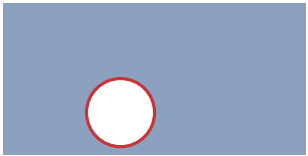
Border

Vật bị cắt bởi mép ảnh vẫn có mask đến mép, không tưởng tượng phần ngoài ảnh.

Hai mảnh nhìn thấy có thể vẫn là một instance



hai component thấy được



lỗ thật trong vật

Cùng instance id có thể trải trên nhiều component nhìn thấy. Chốt riêng: visible-only hay amodal cho vùng bị che; vùng rỗng là hole hay nền.

Tạo mask: công cụ phải theo hợp đồng

Polygon / brush

Vertex nhanh với cạnh thẳng; brush hợp đường cong nhỏ/lỗ. Cả hai phải overlay sau rasterize.

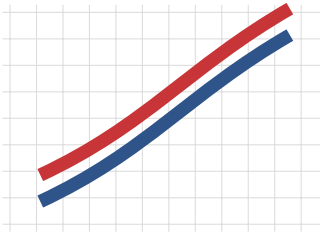
SAM + CVAT

SAM đề xuất; CVAT lưu/sửa mask; export rồi kiểm lại class map và overlay.

Overlap

Chốt visible hay amodal. Panoptic: một owner/pixel; instance masks có thể overlap theo format.

Stress test riêng: vật mảnh lệch hai pixel



Dây/cọc/thanh chắn cong có diện tích nhỏ.

Hai dải gần song song có thể nhìn "đúng dáng" nhưng phần giao rất ít.

Không trộn vào train vùng dập; dùng stress test có target riêng.

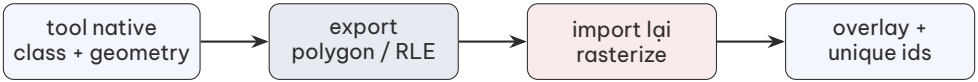
Mask có ba cách lưu phổ biến

Dạng	Mạnh	Bẫy
PNG class-id	đọc trực tiếp, lossless	palette/RGB bị hiểu nhầm thành class
Polygon	nhẹ, dễ sửa điểm	biên cong, lỗ, polygon lỗi
RLE	gọn cho vùng lớn/phức tạp	khó nhìn bằng mắt; quy ước order

Kiểm ở biên

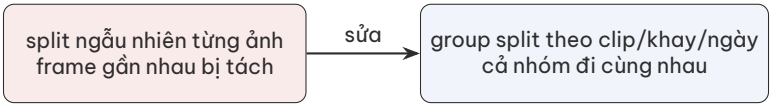
Rasterize rồi overlay lên ảnh thật; đừng chỉ kiểm file parse được.

Export không phải bằng chứng: phải round-trip



Contract tối thiểu: class id/name, image id, $H \times W$, geometry type, coordinate convention, hole/multi-component policy và ignore value.

Rò rỉ: cùng một khay xuất hiện ở hai tập



Model ghi nhớ nền, ánh sáng và đúng quả đó; validation đẹp hơn khả năng dùng thật.

Dự đoán mọi pixel là nền

accuracy $= 9.900/10.000 = 99\%$
IoU của vùng dập $= 0$
Dice của vùng dập $= 0$

Vì vậy pixel accuracy một mình che mất thất bại của lớp nhỏ.

Acceptance audit: dữ liệu vào phải qua cổng

1. Mọi image_id có đúng một mask và ngược lại.
2. H, W khớp; unique values thuộc class schema.
3. Overlay mẫu ngẫu nhiên và toàn bộ ca có lỗ/chạm biên.
4. Không group_id nào xuất hiện ở hơn một split.
5. Báo phân bố diện tích foreground và ca mask rỗng.

Dữ liệu segmentation: ba tầng hiểu

Ý tưởng: Model học đúng thứ mask nói, kể cả khi mask sai.

●○○ **Cơ bản**	Ảnh và mask phải đi thành cặp, cùng kích thước, cùng biến đổi.
●●○ **Nâng cao**	Class id là rời rạc: resize nearest; split theo nhóm để chặn rò rỉ.
●●● **Chuyên sâu**	Hợp đồng còn gồm topology, vùng rỗng, uncertainty ở biên và phân bố foreground theo miền triển khai.

04

Đo Mask Và Audit Thất Bại

Một con số không nói được model sai ở đâu

Bốn vùng sau khi chồng hai mask

	Ground truth 1	Ground truth 0
Dự đoán 1	TP: phần giao	FP: tô thừa
Dự đoán 0	FN: bỏ sót	TN: nền đúng

IoU và Dice của foreground dùng TP, FP, FN; nền lớn không trực tiếp lấn át.

IoU: phần giao chia phần hợp

$$\text{IoU} = \frac{TP}{TP + FP + FN}$$

Ví dụ: $TP = 30$, $FP = 10$, $FN = 20$

$$\text{IoU} = 30/(30 + 10 + 20) = 0{,}50$$

Đọc bằng lời

Trong toàn bộ pixel được mask thật hoặc mask dự đoán phủ, một nửa được hai bên đồng ý.

Dice: giao được đếm hai lần

$$\text{Dice} = \frac{2TP}{2TP + FP + FN}$$

Cùng ví dụ: $2 \cdot 30/(60 + 10 + 20) = 0{,}667$

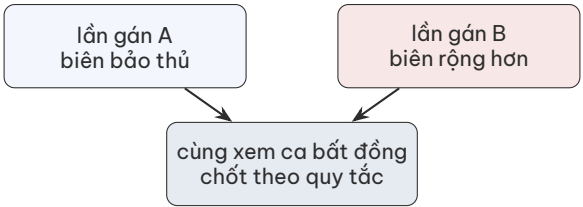
$\text{Dice} = \dfrac{2\,\text{IoU}}{1 + \text{IoU}}$ với cùng hai mask.

Mask rỗng: quy ước của ngày hôm nay

Ground truth / prediction	Cách ghi
rỗng / rỗng	đúng âm tính; không đưa vào mean IoU/Dice foreground
rỗng / có mask	false-positive case; IoU=Dice=0
có mask / rỗng	false-negative case; IoU=Dice=0

Báo thêm: số ca rỗng, accuracy trên ca rỗng, và mean foreground trên ca ground truth không rỗng.

Hai mask theo cùng guideline vẫn có thể lệch nhau



Đo agreement để tìm chỗ guideline mơ hồ; xem overlay bất đồng; adjudicate và cập nhật ví dụ. Không dùng một ngưỡng IoU phổ quát để "chấm người".

Tự tính (12 phút): một dự đoán tô thừa

TP	FP	FN
48	24	8

1) IoU? 2) Dice? 3) Lỗi chính là tô thừa hay bỏ sót?

Đáp án trong ghi chú; hãy viết phép tính trước khi bấm máy.

Macro, micro, weighted: cùng mask, ba câu chuyện

Average	Cách gộp	Khi đọc phải nhớ
Macro	trung bình các lớp	lớp hiếm có tiếng nói ngang nhau
Micro	cộng pixel trước	lớp lớn chi phối
Weighted	cân theo support	có thể che lớp nhỏ

Loss để học không nhất thiết là metric để báo

BCEWithLogitsLoss

So từng pixel; ổn định; có thể dùng `pos_weight` khi foreground hiếm.

Dice loss

Tối ưu overlap toàn mask; cần quy ước smoothing và batch reduction.

Loss trơn để gradient học; metric sau threshold để trả lời về đầu ra sử dụng.

Threshold đổi mask, không đổi logits



Chọn threshold trên validation theo chi phí FP/FN; không chọn trên test.

Cùng lỗi biên 5 px, IoU kể hai câu chuyện



$L = 400, d = 5$: IoU $= 0{,}950625$

$L = 40, d = 5$: IoU $= 0{,}5625$

Sơ đồ không cùng tỉ lệ pixel; bề dày tương đối d/L được phóng để thấy khác biệt.

Ma trận audit: lỗi nào cần sửa ở đâu?

Triệu chứng	Kiểm đầu tiên	Hành động có thể
mask lệch khỏi vật	paired transform	khóa cùng random params
biên răng cưa	resize/export	nearest cho class id
bỏ vùng nhỏ	phân bố size	thêm ca nhỏ, crop, loss phù hợp
đẹp trên val, hỏng thật	group/domain split	thu thập miền triển khai

Failure audit phải giữ cả ca bị từ chối

Với mỗi ảnh test, lưu: image_id, group_id, model id, threshold, IoU/Dice theo lớp, diện tích thật/dự đoán, quyết định thật/dự đoán, và cờ failure mode.

Tập audit có chủ đích

Xem top FP, top FN, worst IoU, vật nhỏ nhất, điều kiện sáng lạ, mask rỗng và ca model không chắc.

Thảo luận (20 phút): điểm cao, quyết định sai

Model A có Dice 0,88 nhưng thường tô thừa quanh biên; Model B có Dice 0,84 nhưng sai số diện tích nhỏ hơn. Hệ thống bật kiểm tra khi tỉ lệ dập vượt ngưỡng.

1. Chọn model nào? Cần thêm số nào?
2. Split nào làm câu trả lời đáng tin?
3. Hai ảnh nào phải mở xem trước?

Đo segmentation: ba tầng hiểu

Ý tưởng: Điểm overlap phải đi cùng ảnh lỗi và hậu quả.

●○○ Cơ bản	IoU/Dice bỏ TN nên không bị nền lớn nâng điểm trực tiếp.
●●○ Nâng cao	Macro, micro và background rule kể ba câu chuyện khác nhau; phải ghi cách gộp.
●●● Chuyên sâu	Region overlap có thể bỏ qua topology và sai biên; thêm slice/boundary/decision metric theo failure mode.

05

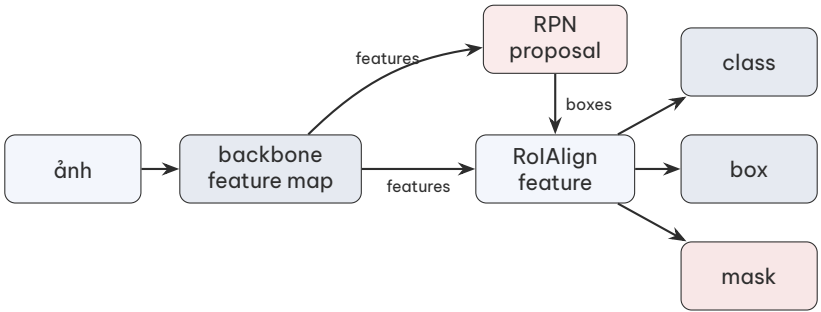
Ba Đường Đến Mask

U-Net từ số 0, model pretrained và promptable segmentation

Ba đường, ba câu hỏi

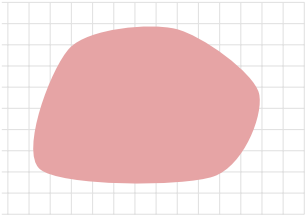
Đường	Ta có sẵn gì?	Câu hỏi chính
Tiny U-Net từ số 0	ảnh + mask của mình	pipeline có đúng và học được?
Pretrained fine-tune	trọng số thị giác + nhãn	chuyển miền có đủ tốt?
Prompt inference	model nền + điểm/box/text	gợi ý mask có cần sửa?

Mask R-CNN: proposal đến mask từng RoI



RPN (mạng đề xuất vùng) tạo box; RoI là vùng quan tâm. RoIAlign lấy feature tại box; ba head trả class, box và mask.

Mask 28×28: thiết kế gốc theo RoI



mask theo RoI, lưới cố định

Bài gốc dùng mask $m \times m$ và báo $m = 28$. Sau đó mask được đặt lại vào box ảnh.

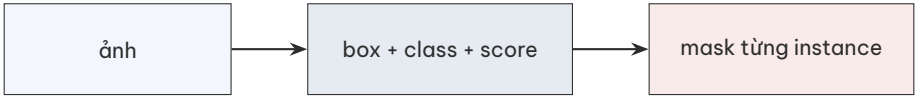
Caveat: vật mảnh/biên nhỏ có thể mất chi tiết.

Từ số 0 nghĩa là trọng số ngẫu nhiên



Không tải backbone, checkpoint hay embedding đã học. Dữ liệu của dự án phải dạy cả cạnh, kết cấu lẫn khái niệm.

YOLO segmentation: thêm mask cho mỗi instance



Pin: `ultralytics==8.4.152`; pretrained checkpoint: `yolo26n-seg.pt`. Fine-tune khi cần mask từng instance.

Fine-tune: giữ kiến thức chung, sửa cho miền mới

1. Bắt đầu từ checkpoint pretrained có license phù hợp.
2. Thay schema lớp theo ứng dụng.
3. Train trên train split; chọn cấu hình bằng validation.
4. So với baseline đơn giản và kiểm miền lạ.

Checkpoint biết “hình ảnh nói chung”, nhưng chưa biết định nghĩa “vùng dập” của dự án.

Fine-tune YOLO: ghim gói và checkpoint

```
pip install ultralytics==8.4.152

from ultralytics import YOLO

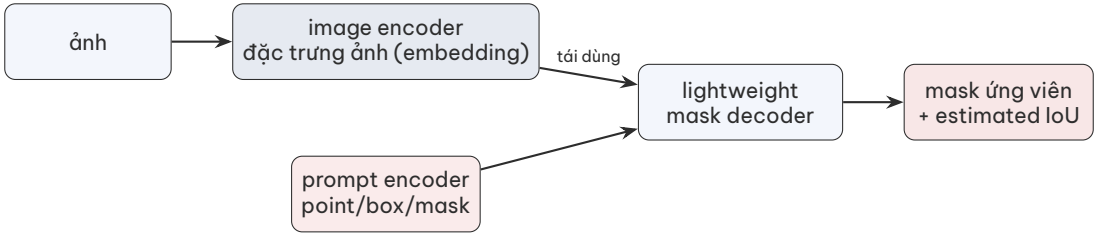
model = YOLO("yolo26n-seg.pt")

model.train(data="dataset.yaml", epochs=20, imgsz=640)

for result in model.predict(source="sample.jpg"):
  masks = result.masks.data if result.masks else None
```

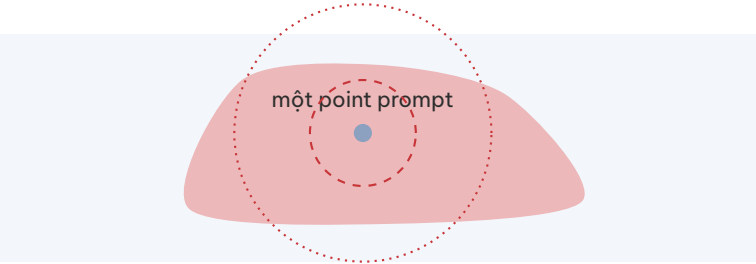
Đây là **pretrained fine-tuning**; không phải random initialization hay prompt inference.

SAM gốc: tính ảnh một lần, đổi prompt nhiều lần



Với **SAM 2023**, embedding ảnh có thể tính một lần; prompt mới chạy prompt encoder + decoder nhẹ.

Một điểm có thể chỉ phần, vật, hoặc cả nhóm



Prompt mơ hồ có thể khớp một phần, toàn vật, hoặc nhóm vật.
Multimask đưa vài giả thuyết; người dùng vẫn chọn theo data contract.

SAM 2 đến SAM 3.1: prompt nào?

SAM 2 (2024)

Promptable segmentation cho ảnh/video; video dùng streaming memory. Điểm/box hướng model tới vật.

SAM 3 (2025), repo 3.1 (2026)

Concept prompt bằng cụm danh từ hoặc exemplar để tìm mọi instance khớp; repo vẫn có visual prompts.

Đây là pretrained prompt inference; mask vẫn qua schema và acceptance.

AI hỗ trợ: đề xuất, kiểm, sửa, ghi lý do



Tốc độ không đủ: cần đo thời gian sửa, tỉ lệ accept nguyên trạng và lỗi còn sót trên gold set.

SAM không thay thế schema

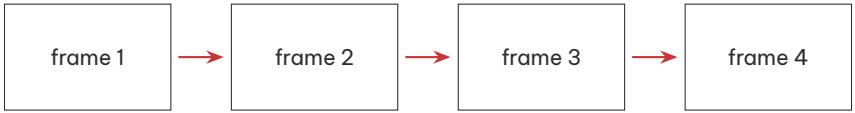
SAM có thể làm

Tách vùng theo prompt; cho nhiều mask ứng viên; giúp dựng nháp nhanh.

Dự án vẫn phải chốt

Class nào tồn tại; biên mơ hồ; lỗ; vật chạm; ca bỏ qua; mức chất lượng chấp nhận.

Prompt propagation: lỗi nhỏ có thể đi xa



Mask sai ở frame đầu có thể trở thành bộ nhớ sai; cần checkpoint audit và prompt sửa.

Chọn đường bằng rủi ro và tài nguyên

	Hiểu sâu	Khởi động	Nhóm nhỏ	Kiểm soát
Tiny U-Net	cao	chậm	yếu	cao
Fine-tune	vừa	nhanh	khá	vừa
Prompt + review	vừa	rất nhanh	phụ thuộc prompt	phải audit

Bảng là khung định tính. License và phần cứng phải kiểm từ repo chính thức trước triển khai.

AI cho segmentation: ba tầng hiểu

Ý tưởng: Pretrained không có nghĩa là biết quy tắc của dự án.

●○○ Cơ bản	Prompt model tạo mask nháp; dữ liệu chuẩn vẫn cần kiểm.
●●○ Nâng cao	Fine-tune chuyển đặc trưng đã học sang miền mới; split và metric giữ nguyên vai trò.
●●● Chuyên sâu	Foundation model thay chi phí khởi tạo, nhưng không xoá domain shift, uncertainty hay acceptance audit.

Chốt buổi sáng: bốn câu phải trả lời

1. Quyết định cần semantic, instance hay panoptic?
2. Tensor contract từ ảnh đến mask là gì?
3. Split có chặn cùng nguồn lọt sang validation/test?
4. Metric nào phản ánh đúng failure mode và quyết định?

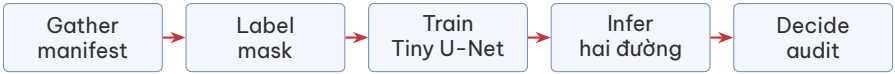
Nếu bốn câu chưa rõ, model mới hơn chỉ làm vòng lặp đắt hơn.

Thực Hành Bốn Giờ

06

Hai baseline, một audit: Tiny U-Net thật và AI hỗ trợ

Mục tiêu lab: một kết luận có bằng chứng



Mỗi nhóm nộp: data contract, split manifest, checkpoint, bảng metric, 6 overlay lỗi và một đề xuất vòng sau.

Trước khi chạy: khóa data contract

Trường	Giá trị nhóm phải điền
Input	RGB, kích thước train, miền giá trị
Target	binary hay multiclass; shape; class ids
Empty mask	hợp lệ hay lỗi
Group key	clip/khay/ngày nào phải đi cùng split
Decision	đại lượng và ngưỡng đánh giá

Trạm 1 — Gather: manifest trước thư mục

```
image_id, image_path, mask_path, group_id, split
tray001_001, images/a.png, masks/a.png, tray001, train
tray002_014, images/b.png, masks/b.png, tray002, val
```

Kiểm đường dẫn tồn tại, id duy nhất, group không cắt qua split.

Trạm 2 — Label: ba lớp kiểm mask

1. **File**: đọc được, lossless, H, W khớp.
2. **Schema**: unique values hợp lệ; mask rỗng theo luật.
3. **Meaning**: overlay đúng biên, lỗ, vùng chạm mép và ca mơ hồ.

AI là đường nháp

Nếu dùng prompt model, lưu prompt và mask trước/sau sửa để đo chi phí sửa.

Dataset: đọc RGB và mask rời rạc


```
def __getitem__(self, i):
  row = self.rows[i]
  image = read_rgb(row.image_path)
  mask = read_gray(row.mask_path)
  image, mask = paired_transform(image, mask)
  mask = (mask > 0).float().unsqueeze(0)
  return image, mask
```

Assert cuối: image $[3, H, W]$, mask $[1, H, W]$, mask unique $\subseteq \{0, 1\}$.

Trạm 3 — train epoch skeleton

```
model.train()
train_loss = 0.0
for images, masks in train_loader:
  images, masks = images.to(device), masks.to(device)
  optimizer.zero_grad()
  logits = model(images)
  loss = criterion(logits, masks)
  loss.backward()
  optimizer.step()
  train_loss += loss.item() * images.size(0)
```

Chia tổng theo số ảnh đã thấy; log seed, epoch, checkpoint và data version.

Validation skeleton: không gradient, có metric

Nâng cao

VINUNIVERSITY

```
model.eval()
meter.reset()
with torch.no_grad():
  for images, masks in val_loader:
    logits = model(images.to(device))
    probs = torch.sigmoid(logits)
    preds = probs >= threshold
    meter.update(preds.cpu(), masks)
if meter.dice > best:
  best = meter.dice
  torch.save({"model": model.state_dict()}, best_path)
```

Khởi tạo `best=-float("inf")` một lần trước vòng epoch; lưu sau cả validation epoch.

Trạm 4 — khóa checkpoint rồi inference

```
state = torch.load(best_path, map_location=device)
model.load_state_dict(state["model"])
model.eval()
with torch.no_grad():
  logits = model(image[None].to(device))
  prob = torch.sigmoid(logits)[0, 0]
  bruise = prob >= threshold
fruit_ref = fruit_ref.to(device).bool()
assert fruit_ref.shape == bruise.shape
assert fruit_ref.sum() > 0
ratio = (bruise & fruit_ref).sum() / fruit_ref.sum()
```

Tính từng ảnh. `fruit_ref` là ROI quả được cung cấp cho bài học hoặc model thứ hai đã đánh giá riêng.

So công bằng: hai đường cùng dự đoán vùng dập

Đường A

Tiny U-Net dự đoán bruise mask; threshold khóa trên validation.

Đường B

Prompt/pretrained cũng tạo bruise mask; prompt policy khóa trên validation.

Mask cả quả chỉ làm mẫu số `fruit_ref`; không so nó với mask vùng dập như cùng target.

Lối cơ bản: làm lab không cần viết model

1. Dùng checkpoint và predictions giảng viên cung cấp.
2. Kiểm manifest, image–mask pairing và group split.
3. Chọn threshold trên validation bằng worksheet.
4. Tính IoU/Dice; dựng sáu overlay audit.
5. So prompt proposal với cùng bruise target và viết kết luận.

Đo thời gian công bằng: thủ công và AI

Nhóm	Nửa đầu	Nửa sau
A	thủ công · tập 1	AI + sửa · tập 2
B	AI + sửa · tập 1	thủ công · tập 2

Khóa quy tắc, target và cổng chấp nhận. Ghi tổng thời gian đến khi mask đạt chuẩn, số lần sửa và lỗi còn lại trên audit độc lập.

Đường	Dice	IoU	lỗi quyết định	**chi phí sửa**
Tiny U-Net	—	—	—	không áp dụng
Pretrained prompt	—	—	—	phút / ảnh

Thêm slice theo vùng nhỏ, ánh sáng lạ, mask rỗng; không chỉ báo mean.

Sáu overlay bắt buộc

1. test tốt nhất;
2. test tệ nhất;
3. FP nhiều nhất;
4. FN nhiều nhất;
5. foreground nhỏ nhất;
6. hai đường bất đồng nhất.

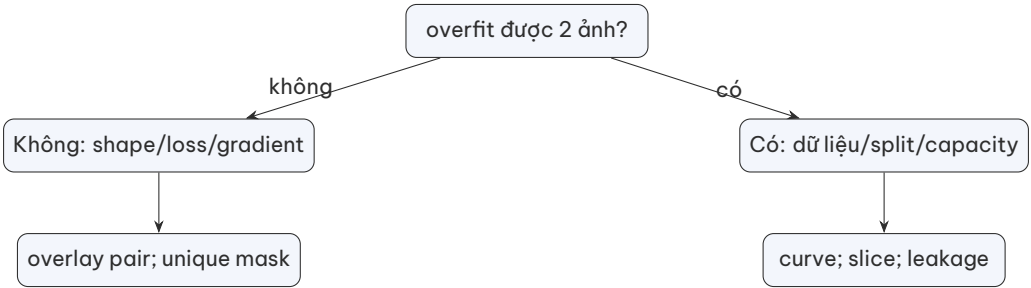
Mỗi overlay có một câu

> “Lỗi này đến từ dữ liệu, transform, threshold, model hay chính data contract?”

Acceptance gate trước khi kết luận

1. Không rò group giữa split.
2. Mọi test inference chạy bằng lựa chọn đã khóa trên validation.
3. Metric ghi include-background, average, empty-mask rule.
4. Kết quả có cả aggregate, slice và overlay lỗi.
5. Quyết định được kiểm riêng, không suy từ Dice.

Nếu model không học: cây chẩn đoán 10 phút



Kế hoạch 240 phút

Khối	Phút	Đầu ra
Chốt contract + manifest	25	schema, group split
Gather + acceptance	35	manifest sạch
Label/AI draft + audit	55	mask + log sửa
Train Tiny U-Net	45	checkpoint + curves
Infer hai đường	35	predictions khóa
Scorecard + overlay	35	metric, 6 lỗi
Kết luận nhóm	10	một đề xuất vòng sau

Mục tiêu: Đi trọn năm trạm từ ảnh mặt bàn đến quyết định có thể audit.

Deliverable: Data contract, manifest, checkpoint Tiny U-Net, scorecard hai đường, sáu overlay lỗi và một đề xuất vòng sau.

Thời gian: 240 phút

Phiếu nộp: báo cáo QC mask sáu ô

1. Data contract một bảng.
2. Sơ đồ group split và bằng chứng không rò.
3. Learning curves + checkpoint được chọn.
4. Scorecard hai đường, có quy ước metric.
5. Sáu overlay lỗi có giả thuyết.
6. Quyết định: chọn đường nào, chưa biết gì, thu thập gì tiếp.

Tổng kết — Key Takeaways

Những ý chính cần nhớ trước khi sang bài tiếp theo

1. Segmentation biến ảnh thành quyết định ở mức pixel; ba loại trả lời ba hợp đồng khác nhau.
2. Binary segmentation: ảnh $[B, 3, H, W]$, logits/mask $[B, 1, H, W]$; logits đi thẳng vào BCEWithLogitsLoss.
3. U-Net nén bằng encoder, trả chi tiết bằng skip connection, rồi giải mã về kích thước gốc.
4. Ảnh–mask phải ghép đúng, mask resize nearest và split theo group.
5. IoU, Dice và failure audit phải nối tới lỗi quyết định; pretrained và SAM không thay schema hay acceptance.

Tài liệu tham khảo (1/3)

- Ronneberger, Fischer, Brox. *U-Net: Convolutional Networks for Biomedical Image Segmentation*. MICCAI, 2015.
- Long, Shelhamer, Darrell. *Fully Convolutional Networks for Semantic Segmentation*. CVPR, 2015.
- He et al. *Mask R-CNN*. ICCV, 2017.
- Kirillov et al. *Panoptic Segmentation*. CVPR, 2019.

Tài liệu tham khảo (2/3)

- Kirillov et al. *Segment Anything*. ICCV, 2023; Ravi et al. *SAM 2*, 2024.
- Carion et al. *SAM 3*, 2025; official repositories checked 16/09/2026.
- PyTorch: `BCEWithLogitsLoss`; Torchvision functional transforms.
- Cheng et al. *Boundary IoU*. CVPR, 2021.

Tài liệu tham khảo (3/3)

- Ultralytics 8.4.152 PyPI + task docs tại tag v8.4.152.
- CVAT docs: Segmentation Mask, COCO polygon/RLE, mask creation caveats.
- Research note Day 5: claim ledger, URL, ngày kiểm và page index.
- Số IoU/Dice, cảnh trái cây và stress test là ví dụ tự dựng.

Re-check model access, license, version và default ngay trước buổi dạy.